

Lab 01 — Hands-on lab: see isolation with your own eyes

Goal: verify every claim of the theory experimentally. By the end you will have seen that a container is a process of your WSL machine, and that the `root` of a rootless container is you.

Prerequisites — Windows 10/11 with WSL 2 and an Ubuntu distribution (22.04 or newer). No files are needed for this lab. The outputs shown were produced with Podman 5.8; from Podman 4.9 the commands are identical, only a few display details vary.

Step 0 — Prepare WSL and install Podman

In a **PowerShell** terminal (Windows):

```
wsl --version          # WSL version 2.x expected
wsl --list --verbose    # your Ubuntu must be VERSION 2
```

Then in the **Ubuntu** terminal:

```
cat /etc/wsl.conf
```

Observe whether it contains `[boot]` followed by `systemd=true`. If not, add it:

```
printf '[boot]\nsystemd=true\n' | sudo tee /etc/wsl.conf
```

then, from PowerShell, `wsl --shutdown` and reopen Ubuntu. Now install Podman:

```
sudo apt update && sudo apt install -y podman
podman --version
```

→ WINDOWS / WSL

WSL 2 is a tiny Hyper-V VM that boots in a second and shares RAM with Windows. By default it has **no** `systemd`: a historical choice by Microsoft, and it is precisely `systemd` that delegates to your user the right to create *cgroups*. Without it, `podman run --memory` and `podman stats` do not work rootless. Hence the step above. You need neither Docker Desktop nor Podman Desktop: here Podman is a plain Ubuntu package. (If you do use Podman Desktop, `podman machine` creates its own WSL distribution and the commands of this lab are identical.)

Step 1 — Identify the engine

```
podman version
```

Observe a single block, `Client: Podman Engine`, with `Version: 5.x.x` and `OS/Arch: linux/amd64`. No `Server` block.

```
podman info | head -n 40
podman info --format 'rootless={{.Host.Security.Rootless}} cgroups={{.Host.CgroupManager}}
network={{.Host.NetworkBackend}} runtime={{.Host.OCIRuntime.Name}}'
```

Observe `rootless=true cgroups=systemd network=netavark runtime=crun`, and in the long output the lines `kernel: 6.6.87.2-microsoft-standard-WSL2`, `idMappings:` (we come back to it in step 5) and `graphRoot: /home/<you>/.local/share/containers/storage`.

Explanation. With Docker, `version` queries two halves — client and daemon — and `info` describes the daemon. With Podman there is one program: `podman info` describes what **your user** can do. The `graphRoot` inside your `home` confirms it: images are not in `/var/lib`, they belong to you.

Step 2 — The first container, and where it went

```
podman run alpine echo "hello from the container"
```

Observe first `Resolved "alpine" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)`, then `Trying to pull docker.io/library/alpine:latest...`, some `Copying blob` lines, `Writing manifest`, the message printed... then an immediate return to the prompt.

PODMAN

Docker silently completes `alpine` into `docker.io/library/alpine`. Podman refuses to guess: it consults a list of known aliases (`alpine`, `nginx`, `debian`, `node`, `postgres` ...) and, for an unknown name, it **asks** you which registry to search — or fails if no terminal is available. That is why company Dockerfiles and scripts always write the full name: `docker.io/library/eclipse-temurin:21-jre`. Get into the habit now.

```
podman ps
podman ps -a
```

Observe that `podman ps` shows **nothing**, but `podman ps -a` shows the container, with a random name (`trusting_sanderson` ...), the image under its full name `docker.io/library/alpine:latest`, and the status `Exited (0)`.

```
podman run --rm alpine echo "this one will leave no trace"
podman ps -a
```

Observe that no new container appears: `--rm` removes the container when it exits.

Explanation. A container lives exactly as long as its main process. `echo` printed one line and exited: the container died with it, but it is not removed — it stays as an inspectable corpse.

`podman ps` only lists running containers.

Step 3 — The kernel is the host's (and the host is WSL)

```
uname -r
podman run --rm alpine uname -r
podman run --rm debian uname -r
```

Observe that all **three** commands print the same value, `6.6.87.2-microsoft-standard-WSL2` for example — even though Ubuntu, Alpine and Debian are three different systems.

```
podman run --rm alpine cat /etc/os-release | head -n 2
podman run --rm debian cat /etc/os-release | head -n 2
```

Observe two different results this time: `Alpine Linux` and `Debian GNU/Linux`.

Explanation. Proof done: the image brings the *userland* (files, binaries, libraries), the kernel comes from the host and is never duplicated. And that host is not Windows: the `microsoft-standard-WSL2` suffix is the signature of the Linux kernel Microsoft compiles for WSL. Your containers run in that VM.

→ LINUX

`/etc/os-release` is a plain text file every distribution installs to introduce itself. `uname -r`, on the other hand, is a **system call**: the answer comes from the kernel. That is why the first varies from one container to the next and the second does not.

Step 4 — See the process from both sides

Start a container that lasts:

```
podman run -d --name watcher alpine sleep 600
podman ps
```

Observe the status `Up`, the name `watcher`, and the command `sleep 600`.

View **from inside**:

```
podman exec watcher ps -o pid,ppid,comm
```

Observe a tiny list: `sleep` has **PID 1**, and your `ps` is PID 2.

View **from the host**:

```
ps -ef | grep "[s]leep 600"
podman inspect --format '{{.State.Pid}}' watcher
```

Observe that the same process exists on the host, owned by **your user**, with an ordinary PID (1854 for example), and that `podman inspect` gives you precisely that PID.

```
podman top watcher
```

Observe `USER root`, `PID 1`, `COMMAND sleep 600`: the "container view" of the same process, reconstructed by Podman.

Explanation. One and the same process, two numberings. Inside, the `pid` namespace makes it believe it is the first process of the system; outside, it is one process among hundreds, and it is yours. That is the whole idea of a container.

Check that this isolation can be removed:

```
podman run --rm --pid=host alpine ps -o pid,comm | head -n 8
```

Observe the processes of **your WSL** (`init`, `systemd`, `common` ...) listed from inside a container.

Explanation. Isolation is an option, not an intrinsic property. That is why `--pid=host` and `--privileged` are forbidden by default in production. Note `common` in passing: it is the small supervisor Podman leaves behind each container, since there is no daemon to do it.

Step 5 — The root that isn't one (rootless)

```
podman exec watcher id
```

Observe `uid=0(root) gid=0(root)`: inside the container, `sleep` runs as root.

```
podman top watcher user,huser,pid,hpid,comm
```

Observe:

USER	HUSER	PID	HPID	COMMAND
root	1000	1	1854	sleep 600

`USER` is the identity as seen from the container, `HUSER` the real identity on the host: `1000` is you (`id -u` to check).

```
podman unshare cat /proc/self/uid_map
```

Observe a mapping table of this kind:

0	1000	1
1	100000	65536

Explanation. That is the `user` namespace in action. Line 1: the container's UID `0` **is** your UID `1000`. Line 2: the container's UIDs `1` to `65536` are mapped onto a "spare" range of UIDs (`100000` +, defined in `/etc/subuid`) that nobody else uses. So the container's "root" has, on the host, only your rights. A compromised container cannot become root on your WSL: there is nothing to escalate to.

→ SECURITY

With Docker, the daemon runs as root and a container `root` is, unless specially configured, the real root of the host. Isolation then relies only on the `pid / mnt / net` namespaces and on the *capabilities* that were dropped. Rootless Podman adds a layer Docker does not have by default: even if everything else gives way, the attacker is an ordinary user.

Step 6 — Immutable image, disposable container

```
podman run -d --name c1 alpine sleep 600
podman run -d --name c2 alpine sleep 600
podman exec c1 sh -c 'echo "data from c1" > /mark.txt'
```

Check that writes are isolated:

```
podman exec c1 cat /mark.txt      # prints: data from c1
podman exec c2 cat /mark.txt      # cat: can't open '/mark.txt': No such file or directory
```

Check that the image itself did not move:

```
podman run --rm alpine ls /mark.txt  # No such file or directory
```

Measure that layer:

```
podman ps -s --format 'table {{.Names}}\t{{.Size}}'
```

Observe a size like `11.4kB (virtual 8.72MB)`: `virtual` is image + layer, the first value is what the container consumes **on its own** — a few kilobytes of metadata, plus your file.

Finally, destroy and start over:

```
podman rm -f -t 0 c1
podman run -d --name c1 alpine sleep 600
podman exec c1 ls /mark.txt      # No such file or directory
```

Explanation. `podman rm` destroys the container **and** its writable layer. The new `c1` starts again from the exact state of the image. Any data to keep must leave the container: that is the subject of lab 06.

PODMAN

Why `-t 0`? `podman rm -f` starts with a polite stop (`SIGTERM`), waits **10 seconds**, then kills. Docker kills immediately. Since `sleep` ignores `SIGTERM` (lab 03), without `-t 0` you would wait ten seconds staring at a warning `StopSignal SIGTERM failed to stop container ... resorting to SIGKILL`. That is not a bug: it is Podman telling you that your application does not stop cleanly.

Step 7 — Cgroups, or the consumption limit

```
podman run -d --name limited --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limited
```

Observe the `MEM USAGE / LIMIT` column: `471kB / 134.2MB`, not the total RAM of your machine.

Compare with a container without a limit:

```
podman stats --no-stream watcher
```

Observe that the limit shown is the total RAM... **of the WSL VM**, for instance `7.7GB` on a 16 GB PC.

Explanation. Without `--memory`, a container can consume all available memory. The namespace protects nothing here: the cgroup is what caps. If this step fails with `OCI runtime error: ... cgroup ...`, `systemd` is not active in your WSL (step 0).

→ WINDOWS / WSL

By default WSL 2 sees only **50% of Windows' RAM** (and at most 8 GB on older versions). It is adjustable in `%UserProfile%\wslconfig` (`[wsl2]` then `memory=12GB`). When a container "runs out of memory" on a Windows workstation, the limit that matters is often that one, not the container's.

Step 8 — inspect, the source of truth

```
podman inspect watcher | head -n 30
```

That is verbose: target what you want with a *Go template*.

```
podman inspect --format '{{.State.Status}}' watcher
podman inspect --format '{{.Config.Image}}' watcher
podman inspect --format '{{json .Config.Cmd}}' watcher
podman inspect --format '{{.NetworkSettings.IPAddress}}' watcher
```

Observe respectively `running`, `docker.io/library/alpine:latest`, `["sleep","600"]` ... and **an empty line** for the IP address.

```
podman exec watcher ip -4 addr show eth0
```

Observe that the container nevertheless has an `eth0` interface, with **the same IP address as your WSL** (`172.2x.x.x`).

Explanation. Rootless, an ordinary user is not allowed to create a network bridge. Podman therefore uses `pasta`, a user-space translator that *copies* the host's address into the container; there is no "container IP" to display. That will be a theme of lab 07. For now remember that the empty value is not an error, and that `--network podman` would give you a real bridge with a `10.88.0.x` IP:

```
podman run -d --network podman --name bridged alpine sleep 600
podman inspect --format '{{.NetworkSettings.Networks.podman.IPAddress}}' bridged
```

Observe `10.88.0.2`. Compare with the **image** metadata:

```
podman image inspect --format '{{json .Config.Cmd}}' alpine
podman image inspect --format '{{.Architecture}}/{{.Os}}' alpine
```

Observe that the image also carries a default command (`["/bin/sh"]`), which your `sleep 600` overrode at `run`, and `amd64/linux`.

Explanation. `podman inspect` works on **every** object (container, image, volume, network) and gives the real state, without formatting. When the documentation and reality disagree, `inspect` is right.

Step 9 — The CLI, long form, short form... and `docker`

```
podman container ls -a
podman ps -a
podman image ls
podman images
```

Observe identical outputs, pairwise.

```
podman container ls --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

Now pretend to be Docker:

```
alias docker=podman
docker ps
docker images
```

Observe that everything works. To make the alias permanent: `echo 'alias docker=podman' >> ~/.bashrc`. On Ubuntu the `podman-docker` package does the same thing (it provides a `docker` binary that calls `podman`).

Explanation. `--format` accepts a Go template and makes outputs usable in scripts — far more reliable than slicing the default table with `awk`. As for the alias: CLI compatibility is a promise of Podman, and it is what lets you follow any Docker tutorial.

Clean-up

```
podman rm -f -t 0 watcher c1 c2 limited bridged
podman ps -a
```

The `Exited` container from step 2 remains. Remove it by name:

```
podman ps -a --filter status=exited --format '{{.Names}}'
podman rm <name>
```

And if you want to reclaim the space of the Debian image, which will not be used again:

```
podman images
podman rmi debian          # we keep alpine for the next labs
```

PITFALL

you will see `podman container prune`, `podman image prune -a` and `podman system prune -a` everywhere. These commands do not remove "what you just did" but **everything that is not in use**: the images and containers of your other projects go with it. Always remove by name. We will cover `prune` properly in lab 10.

What you must be able to state now

- The kernel shown inside a container is the host's — here, WSL 2's.
- A container's process exists in the host's `ps`, under **your** user — you saw it, with its PID.
- The `root` of a rootless container is a projection of your UID: `podman unshare cat /proc/self/uid_map` proves it.
- A write inside a container reaches neither the image nor the other containers.
- `podman rm` destroys data; `podman stop` does not. `podman rm -f` waits 10 s without `-t 0`.
- Without `--memory`, the only limit is the RAM of the WSL VM.
- `podman inspect --format` is your first diagnostic reflex — and an empty IP rootless is normal.